CS108 Project

WhatsApp Group Wrapped

Joshmitha 241098
Akshit 23b0945

1 Project Overview

WhatsApp Group Wrapped is a browser-based slideshow that shows social dynamics found in a WhatsApp group chat. The idea is similar to Spotify Wrapped. You pick a chat, run two Python scripts, and the browser shows animated slides with stats about each member.

The project has three stages. First, `chat.py` generates a fake WhatsApp chat with eight members who each have a distinct personality. Second, `analyze.py` reads that chat and computes all the statistics. Third, the front end (`index.html`, `style.css`, `app.js`) reads `data.json` and shows the stats as a slideshow. The three stages are independent. The Python scripts communicate through plain text files.

2 Code Structure

- **generator/chat.py**
  Generates a synthetic WhatsApp chat and writes it to `chat.txt`. It defines eight member profiles, each with trait values, hour weights, streak settings, and a frequency weight. The main loop advances the clock, picks a member using weighted random selection, generates a message from the vocabulary, and handles streak behavior.

- **parser/analyze.py**
  Reads `chat.txt` and computes all 11 statistics. Each stat is a separate function. After computing everything, the script builds a single `data` dictionary and writes it to `data.json`.

- **web/index.html**
  The main HTML page. It loads `style.css` and `app.js`. The actual slide content is injected by JavaScript so the HTML file itself is mostly just a shell with a `#slides` container and a navigation bar.

- **web/style.css**
  All visual styles. It defines CSS custom properties for each member's color, the slide transition animations, the bar chart and podium layout, the bubble chart, the hour-of-day bar chart on profile slides.

- **web/app.js**
  The slideshow engine. The `boot()` function fetches `data.json` and calls `buildAllSlides()` which returns an array of slide objects. Each slide object has a `getHTML()` function and an optional `onEnter()` function that runs animations after the slide becomes visible. Navigation works by keyboard arrows, swipe, and clicking the slide.

- **vocabulary.txt**
  A comma-separated list of words used by the chat generator to build messages. Words are picked randomly to form each message.

- **chat.txt**
  The generated chat file. Each line has the format `DD/MM/YY, HH:MM - Name: Message`. This file is the input to `analyze.py`.

- **data.json**
  The output of `analyze.py`. This file is fetched by the browser to render the slideshow. The schema is described in the next section.

3 data.json Schema

The file has two top-level keys. `total_messages_group` is a single integer for the whole chat. `stats` holds per-person dictionaries keyed by member name. There is also a `derived` block with pre-sorted arrays and bar chart percentages so the JavaScript does not have to compute them.

```
{
  "total_messages_group": xxxx,
  "stats": {
    "total_messages":      { "Name": 120 },
    "word_count":          { "Name": 843 },
    "night_owl":           { "Name": 14 },
    "ghost": {
      "ghosted_counts":    { "Name": 20 },
      "msg_counts":        { "Name": 120 },
      "ghost_percentage":  { "Name": 16.67 }
    },
    "conversation_starter": { "Name": 8 },
    "most_used_emoji":     { "Name": [["emoji", count], ...] },
    "busiest_day":         ["22/07/24", 87],
    "longest_silence":     { "hours": 14.5, "days": 0.6,
                             "start": "...", "end": "..." },
    "avg_response_time":   { "Name": 4.5 },
    "hype_person":         { "Name": 2.3 },
    "conversation_killer": { "Name": { "kills": 12,
                                       "total": 120,
                                       "score": 10.0 } },
    "time_of_day":         { "Name": [0, 0, 1, 3, ...] }
  }
}
```

The `time_of_day` field is a list of 24 integers, one per hour. It is used to draw the activity bar chart on each member's profile slide.

4 Installation and Running

Dependencies

Install the two external Python packages before running anything:

```
pip install emoji numpy
```

All other imports (`datetime`, `json`, `random`, `sys`) are part of the Python standard library.

Step 1: Generate the chat

```
python generator/chat.py vocabulary.txt
```

This reads `vocabulary.txt` and writes `chat.txt`. The chat covers July 2024 to November 2024 and has around messages across the eight members.

Step 2: Parse the chat

```
python parser/analyze.py chat.txt
```

This reads `chat.txt` and writes `data.json` with all computed statistics. A confirmation message is printed when done.

Step 3: Start a local server

```
python -m http.server 8000
```

A local server is needed because the browser blocks `fetch()` on `file://` URLs. Run this command from the project root.

Step 4: Open the slideshow

Open `http://localhost:8000/web/` in any browser. Use arrow keys, click, or swipe to move between slides.

5 External Libraries Used

- **numpy** (chat.py)
  Used for weighted random selection of which group member sends the next message.
- **emoji** (analyze.py)
  Used to check whether a word in a message is an emoji. We use `EMOJI_DATA` to handle cases where multiple emojis are joined together as one token.
- **datetime** (chat.py and analyze.py)
  Used to parse message timestamps and find the time difference between messages.
- **json** (analyze.py)
  Used to write all the computed stats into `data.json`.
- **random** (chat.py)
  Used to generate random message lengths, streak lengths, and time gaps between messages.
- **sys** (analyze.py and chat.py)
  Used to read command line arguments like the vocabulary file path.

6 Chat Generator (chat.py)

The file `chat.py` generates a fake WhatsApp group chat. It simulates eight users, each with a different personality. The chat runs from July 2024 to November 2024.

1. Vocabulary-based Message Generation

The function `load_vocabulary()` reads words from `vocabulary.txt`. Messages are made by randomly picking words from this list.

2. Personality-driven Users

Each user is stored as a dictionary with the following fields:

- Name and personality type
- Trait values like chatterbox, hype, emoji, dry, essay, ghost, lurker, conversation_starter, night_owl
- A frequency weight that controls how often this user is picked to send a message
- A list of 24 hour weights that decide when the user is active during the day
- Min and max message length
- Response delay range
- Streak probability and streak length range
- Post streak silence (only for the Ghoster)

The eight members and their personalities are:

- **Aryan** is The Chatterbox. He has a frequency weight of 5.0 which is five times higher than most others. He sends short messages of 2 to 5 words and is active throughout the day.
- **Akshit** is The Night Owl. He is completely inactive from 7 AM to 4 PM. His activity starts picking up from 8 PM and peaks at 1 AM with the highest hour weight of 3.5. The night owl stat counts messages between midnight and 4 AM which is exactly when Akshit is most active.
- **Yash** is The Dry Texter. His message length is 1 to 3 words and because his dry trait is 0.95, he always sends the minimum length. He rarely uses emojis.
- **Shreya** is The Essay Writer. She sends messages of 20 to 60 words and sometimes even longer. Her response delay is 7 to 30 minutes so she is slow to reply. She does use emojis often.
- **Dev** is The Ghoster. He has a streak probability of 0.70 and sends 4 to 7 messages in a burst. After each burst he is forced to stay silent for 2 to 8 hours using a post streak silence setting.
- **Kavya** is The Lurker. Her frequency weight is 0.3 which is the lowest of all members. She sends very few messages and almost never starts a conversation.
- **Joshmitha** is The Conversation Starter. Her conversation_starter trait is 0.95 and she gets a large weight boost after a long silence. Her response delay is only 2 to 6 minutes.
- **Tanvi** is The Emoji Queen and Hype Person. Her emoji trait is 0.95 so almost every message has emojis. Her response delay is 0 to 2 minutes which is the fastest of all members.

3. Weighted User Selection

The function `get_next_member()` picks who sends the next message using `numpy.random.choice` with weights. The weight for each user is calculated based on:

- Their base frequency weight multiplied by the current hour weight
- A boost if they are a hype person and the chat is currently active
- A large boost for conversation starters when there has been silence for 60 or more minutes
- A reduction if the user has already sent too many messages in their time window
- A reduction if this user sent the last message in the chat

4. Streak Behaviour

After a user sends a message, there is a chance they send more messages in a row. This is controlled by the streak probability. The number of messages in a streak is picked randomly between the min and max streak values. Dev has the highest streak probability of 0.70 and sends 4 to 7 messages per burst. After his streak, the `silent_until` dictionary blocks him from sending for 2 to 8 hours.

5. Time Gaps Between Messages

The function `time_gap()` decides how much time passes between messages:

- 5% chance of a long gap of 1 to 48 hours
- 25% chance of a medium gap of 5 to 60 minutes
- 70% chance of a quick reply using the member's own response delay range

6. Output

The chat is saved to `chat.txt`. Each line follows this format:

```
DD/MM/YY, HH:MM - Name:  Message
```

The script takes the vocabulary file as a required command line argument and an optional output file path.

7 Statistics Parser (analyze.py)

The file `analyze.py` reads `chat.txt` and computes all the statistics. The results are saved to `data.json` which the front end reads.

Statistics Implemented

- **Stat 1: Total Messages**
  Counts how many messages each user sent.
- **Stat 2: Word Count**
  Counts total words per user. Emojis that appear as separate tokens are also counted as words.
- **Stat 3: Night Owl**
  Counts messages sent by each user between midnight and 4 AM. The user with the highest count is the Night Owl.
- **Stat 4: Ghost Report**
  For each message, if the next message from a different user comes more than 10 minutes later, the original sender gets one ghost count. The ghost score is the percentage of that user's messages that were ghosted.
- **Stat 5: Conversation Starter**
  Counts how many times each user sent the first message after a silence of 60 or more minutes. The user with the most counts is the Conversation Starter.
- **Stat 6: Most Used Emojis**
  Finds the top 3 emojis used by each person. Each word in the message is checked using `emoji.EMOJI_DATA` to handle cases like where two emojis are joined as one token.
- **Stat 7: Busiest Day**
  Finds the date with the most total messages.
- **Stat 8: Longest Silence**
  Finds the biggest time gap between any two consecutive messages in the chat. It reports the gap in hours and days and also shows the start and end timestamps.
- **Stat 9: Average Response Time**
  For each user, we collect all the cases where they replied to a different user within 60 minutes. The median of those times is their response time. This tells us how fast each person actually replies.
- **Stat 10: Hype Person**
  Same idea as above but uses the mean instead of the median. The user with the lowest mean reply time is the Hype Person.
- **Stat 11: Conversation Killer** (*custom*)
  For each message, we check if the next message in the chat arrives within 60 minutes. If not, that message is counted as a kill. The killer score is the number of kills divided by total messages sent, multiplied by 100. The user with the highest score is The Killer.

Personality Labels

After computing all stats, each user is given a personality label based on where they rank:

- **The Chatterbox** gets the label if they have the highest total message count
- **The Dry Texter** gets the label if they have the lowest words per message
- **The Essay Writer** gets the label if they have the highest words per message
- **The Night Owl** gets the label if they have the most messages between midnight and 4 AM
- **The Lurker** gets the label if they have the lowest total message count

- **The Emoji Queen** gets the label if they used the most emojis overall
- **The Hype Person** gets the label if they have the lowest mean reply time among active members
- **The Conversation Starter** gets the label if they broke the most silences
- **The Ghoster** gets the label if they have the highest ghoster score
- **The Conversation Killer** gets the label if they have the highest conversation killer score

8 Custom Statistics

Custom Stat 1: Conversation Killer

A message is called a conversation killer if no one sends a message within 60 minutes after it. The killer score is:

$$\text{Killer Score} = \frac{\text{Number of killing messages}}{\text{Total messages sent}} \times 100$$

We added this because normal tools only tell you how many messages someone sent or how fast they reply. They do not tell you who keeps ending the conversation. Someone can be very active but still kill the chat often. This metric shows that.

The 60 minute threshold also connects nicely with the conversation starter stat. The killer creates the silence and the starter breaks it.

Custom Stat 2: Ghoster Score

We define a burst-then-vanish event as a pair of consecutive gaps where the first gap is 2 minutes or less (the person is sending in a rapid burst) and the next gap is 60 minutes or more (the person has disappeared). The ghoster score counts how often this pattern happens:

$$\text{Ghoster Score} = \frac{\text{burst-then-vanish events}}{\text{total messages}} \times 100$$

A high score means the user fires messages rapidly and then goes silent. Dev has `min_streak = 4` and `post_streak_silence = (120, 480)` in his profile, so he always produces this pattern. Other members may burst occasionally but they do not vanish for hours afterwards, so their score stays low.

We collect each person's own message timestamps in a separate list and compute gaps from that list. This is important because other members send messages between Dev's burst, so looking only at consecutive same-sender lines in the file would miss the post-burst silence entirely.

9 Definition Choices

- **Direct reply.** We define a direct reply as the next message from a different user within 10 minutes. We picked 10 minutes because that matches the pace of an active chat. A longer window would not separate fast and slow responders very well.
- **Long silence.** A long silence is any gap of 60 or more minutes with no messages at all. We chose 60 minutes because it felt like a natural break in a day's conversation. Shorter values like 15 minutes created too many false starts.
- **Busy day.** The busiest day is simply the day with the most total messages. We thought about measuring message density per hour but total count felt more natural and easier to understand.
- **Response time window.** We only count a reply if it comes within 60 minutes. Anything longer is probably a new conversation and not a direct response. This also stops one overnight silence from pulling the median to a very high value.
- **Ghost percentage window.** A message is ghosted if no different user replies within 10 minutes. This matches our direct reply definition. Each message gets a yes or no and the ghost percentage is how many of the user's messages got a no.

10 Hurdles Faced and How We Overcame Them

- **Building the front end from scratch.** We had no framework and had to figure out CSS animations, transitions, and touch events all separately. We looked up each thing on MDN and tested small pieces before putting them together.
- **Response time was measuring the wrong thing.** The code was adding the reply gap to `times[prev_sender]` instead of `times[sender]`. This meant we were measuring how fast others reply to you, not how fast you reply to others. Changing the key fixed it.
- **Personality badges use exact equality.** If two users have exactly the same max score they both get the badge. We kept this as is for now and noted it as something to improve later.

11 What We Would Do Differently With 10 More Hours

- **Compare two profiles side by side.** The spec says users might want to see multiple profiles at once. Right now you can only view one profile at a time. We would add a mode where you pick two members and their stats appear next to each other.
- **Make messages look more like sentences.** Right now messages are just random words from the vocabulary file. It would be better if they followed a simple structure like subject verb object so they read more naturally.
- **Make the layout work on mobile.** The current design is only tested on a laptop. On a phone some slides go off screen. We would add media queries to fix the layout for smaller screens.
- **Single command to run everything.** Right now you need four separate commands to go from vocabulary file to browser. We would make one script that runs all the steps and opens the browser automatically.

References

[1] MDN Web Docs. *Using CSS custom properties (variables).* `https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties`

[2] MDN Web Docs. *backdrop-filter CSS property.* `https://developer.mozilla.org/en-US/docs/Web/CSS/backdrop-filter`

[3] MDN Web Docs. *cubic-bezier() CSS function.* `https://developer.mozilla.org/en-US/docs/Web/CSS/easing-function/cubic-bezier`

[4] RileyAdair. *Slideshow Vanilla JS with CSS Transition.* CodePen. `https://codepen.io/RileyAdair/pen/ZvyYzr`

[5] MDN Web Docs. *Using Touch Events.* `https://developer.mozilla.org/en-US/docs/Web/API/Touch_events/Using_Touch_Events`

[6] MDN Web Docs. *Using CSS animations.* `https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_animations/Using_CSS_animations`

[7] CSS-Tricks. *Making Charts with CSS.* `https://css-tricks.com/making-charts-with-css/`

[8] Tobias Ahlin. *Common Flexbox Patterns.* `https://tobiasahlin.com/blog/common-flexbox-patterns/`

[9] The Python emoji Library. *emoji 2.x documentation.* `https://pypi.org/project/emoji/`

[10] NumPy Developers. *numpy.random.choice documentation.* `https://numpy.org/doc/stable/reference/random/generated/numpy.random.choice.html`